

Engenharia de Software
Qualidade e Teste de Software

Relatório: Análise de Eficácia de Testes com Teste de Mutação

Lucas Flor Vilela
Matrícula: [Insira sua Matrícula]

Belo Horizonte
2026

1 Análise Inicial

Nesta etapa inicial, foi realizada a medição da cobertura de código convencional utilizando o framework Jest e a primeira execução da ferramenta StrykerJS para obter a pontuação de mutação base.

- **Cobertura de Código Inicial (Jest):** 98,64%
- **Pontuação de Mutação Inicial (Stryker):** 73,71%

Discussão: Observou-se uma discrepância significativa entre os valores. Enquanto a cobertura de código indicava que quase 99% das linhas eram executadas pelos testes, a pontuação de mutação revelou que 26,29% dos bugs introduzidos passariam despercebidos. Isso confirma que a cobertura de código é uma métrica de execução, mas não de eficácia, evidenciando a necessidade de testes que validem o comportamento lógico e não apenas a passagem pelas linhas.

2 Análise de Mutantes Críticos

2.1 Mutante 1: Função Fatorial (Operador Lógico)

O StrykerJS alterou a linha 19 da função `fatorial` trocando o operador `||` (OR) por `&&` (AND).

- **Código Mutado:** `if (n === 0 && n === 1) return 1;`
- **Análise:** Este mutante sobreviveu pois a alteração criou uma condição impossível. Contudo, como o código subsequente inicializava o resultado em 1 e ignorava o laço para valores baixos, o retorno permanecia correto. Isso caracterizou um código redundante, onde a solução foi a remoção da linha ineficaz.

2.2 Mutante 2: Função Raiz Quadrada (Operador Relacional)

A ferramenta alterou a validação de `n < 0` para `n <= 0`.

```

16 function restoDivisao(dividendo, divisor) { return dividendo % divisor; }
17 function fatorial(n) {
18   if (n < 0) throw new Error('Fatorial não é definido para números negativos.');//●●●
19 - if (n === 0 || n === 1) return 1; //●▼●●
+ if (n === 0 && n === 1) return 1;
20   let resultado = 1;
21   for (let i = 2; i <= n; i++) { resultado *= i; }
22   return resultado;
23 }
24 function ... (omitted) ...

```

LogicalOperator Survived (19:7)

Figura 1: Mutante de operador lógico na função fatorial.

- **Análise:** O mutante sobreviveu devido à ausência de um teste para o valor exato zero. Embora a raiz de zero seja zero, o código mutado lançava uma exceção para essa entrada. A falta de um teste de limite (*boundary value*) permitiu que essa falha lógica fosse ignorada.

```

function potencia(base, expoente) { return Math.pow(base, expoente); }
function raizQuadrada(n) {
- if (n < 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');//●▼●
+ if (n <= 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');//●▼●
  return Math.sqrt(n);
}

```

Figura 2: Mutante de operador relacional na raiz quadrada.

2.3 Mutante 3: Conversão de Temperatura (Operador Aritmético)

Na função `fahrenheitParaCelsius`, a multiplicação foi alterada para divisão.

- **Análise:** Como o teste utilizava apenas o valor 32, o resultado da subtração era zero. Matematicamente, zero multiplicado ou dividido por qualquer valor resulta em zero, mascarando a alteração do operador. O teste falhava em validar a fórmula de conversão real.

3 Solução Implementada

Para atingir a meta de eficácia, foram adotadas as seguintes estratégias:

```

91 function isDivisivel(dividendo, divisor) { return dividendo % divisor === 0; }
92 function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; } ●●
93 - function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5/9; } ●▼
  + function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) / 5/9; }
94 function inverso(n) {
95   if (n === 0) throw new Error('Não é possível inverter o número zero. '); ●
96   return 1 / n;
97 }
98

```

ArithmeticOperator Survived (93:53)

Figura 3: Mutante aritmético na conversão de temperatura.

1. **Refatoração de Código Redundante:** Funções como `fatorial`, `clamp` e `produtoArray` foram simplificadas para remover verificações desnecessárias que geravam mutantes equivalentes.
2. **Inclusão de Testes de Borda:** Foram adicionados casos de teste para valores nulos, negativos e limites exatos (ex: `raizQuadrada(0)`).
3. **Validação de Resultados Falsos:** Foram incluídas asserções para garantir que funções booleanas retornem `false` quando esperado, e não apenas `true`.
4. **Dados Diversificados:** Substituíram-se entradas que zeravam equações por valores que forçam o processamento completo das fórmulas matemáticas.

4 Resultados Finais

Após as intervenções, a pontuação final foi:

- **Mutation Score Final:** 98,6%

O aumento de 73,71% para 98,6% comprova a eliminação de quase todas as fragilidades lógicas anteriormente presentes na suíte de testes.

5 Conclusão

O teste de mutação demonstrou ser uma ferramenta superior à cobertura de código para garantir a confiabilidade do software. Ele forçou a análise

de cenários de exceção e limites que são frequentemente ignorados no desenvolvimento focado apenas em funcionalidades. O resultado final de 98,6% assegura que a suíte de testes agora é capaz de detectar alterações sutis na lógica do sistema, proporcionando uma rede de segurança real para futuras manutenções.